План разработки ПО

Проект: Система автоматизации управления жизненным циклом релиза ПО

Версия 1 от 13.11.2026

Авторы: Заранкович Д. Н. (капитан), Брылкина К. К., Поминова А. А., Пономаренко А. А., Карцева П. А., Лазарев Я. О., Деньков Д. С., Чернов А. В.

Курс «Технология программирования кибернетических систем: управление проектами» (ТПКС) · преподаватель Козырев В. П. · НИЯУ МИФИ, 2026

Содержание

1 Определения и термины **2**

2 Жизненный цикл **3**

2.1 Разработка требований 4

2.2 Проектирование 4

2.3 Кодирование 4

2.4 Интеграция 5

3 Организация и роли **5**

4 Среда разработки **6**

4.1 Документация требований 6

4.2 Проектирование 6

4.3 Разработка 6

4.4 Интеграция, сборка и загрузка 6

5 Стандарты **6**

Лист регистрации изменений **7**

1 Определения и термины

План описывает разработку системы автоматизации управления жизненным циклом релиза ПО. Он нужен автору, проверяющему и ответственному за выпуск, чтобы по результату одного процесса можно было начать следующий. Основания плана — системные требования проекта, План УК ПО и Стандарт на разработку требований к ПО. Для процесса требований использована схема окружения со слайда 18 В. П. Козырева, для интеграции — конспект Б21-504, строки 484–526.

ПО — программное обеспечение: программы и связанные с ними данные, предназначенные для выполнения задач вычислительной системой (источник: наше определение; сокращение из гайда старших курсов).

ЖЦ ПО — жизненный цикл ПО: совокупность процессов, через которые ПО проходит при создании и использовании (источник: конспект, К:19–27, 132; наша формулировка).

ВУ — высокий уровень требований: требования к поведению и свойствам ПО, полученные из системных требований и интерфейсов (источник: слайды 17–21, 38).

НУ — низкий уровень требований: требования к компонентам ПО, получаемые при проектировании и используемые при кодировании (источник: слайды 28, 38; конспект, К:71).

СПТ — спецификация программных требований: документ с требованиями к ПО, его входам, выходам, режимам и внешнему взаимодействию (источник: слайды 18–21; сокращение из гайда старших курсов).

ЕК — единица конфигурации: данные жизненного цикла ПО, рассматриваемые при управлении конфигурацией как одно целое и определяемые уникальным именем и версией (источник: слайды 52–53).

Базовая версия — утверждённая версия единицы конфигурации, служащая основой последующей работы. Промежуточные версии базовыми не считаются (источник: слайды 32, 75–76).

СП — сообщение о проблеме: запись об одном несоответствии результатов исходным данным, данных стандартам или процессов планам (источник: слайды 64–66).

ЗИ — запрос на изменение: описание планируемого изменения одной ЕК, порождённое утверждённым сообщением о проблеме (источник: слайды 73–74).

УК ПО — управление конфигурацией ПО: учёт состава, версий и изменений данных жизненного цикла для сохранения целостности ПО (источник: слайды 49, 89).

Интеграция — процесс получения исполняемого объектного кода из результатов кодирования и его загрузки в целевую вычислительную среду (источник: конспект, К:484–497).

Артефакт сборки — файл или набор файлов, полученный процессом интеграции и сохранённый для проверки или выпуска (источник: наша формулировка).

Верификация — проверка соответствия результатов работы её исходным данным и установленным требованиям (источник: конспект, К:97, 161; наша формулировка).

ФИ — формальная инспекция: рассмотрение объекта по установленной процедуре с регистрацией обнаруженных несоответствий (источник: конспект, К:197–202, 258; наша формулировка).

2 Жизненный цикл

Команда последовательно фиксирует требования, проект, исходный код и результат интеграции. Такая модель позволяет проследить, какая версия требований определила поведение испытуемого выпуска. Если проверяющий находит ошибку раннего этапа, он регистрирует СП. Совет решает, какие ЕК затронуты. На каждую изменяемую ЕК открывают ЗИ по Плану УК. После исправления повторяют проверки зависимых результатов.

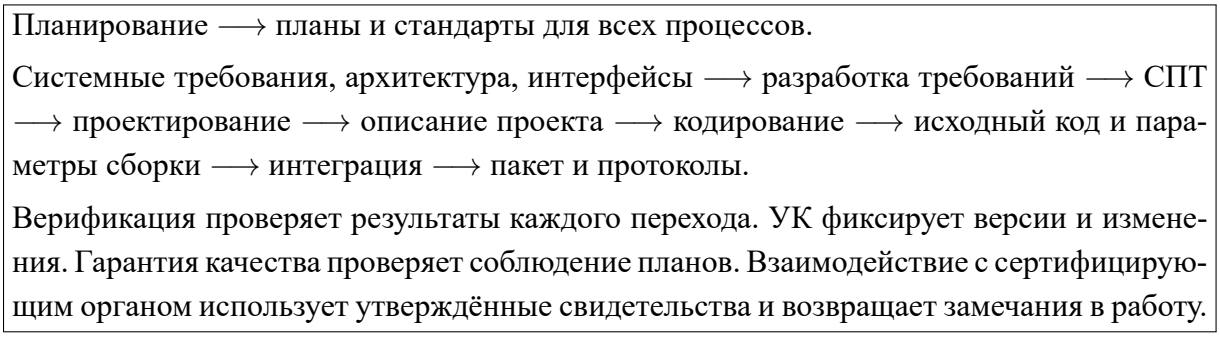


Рисунок 1 – Передача результатов и поддерживающие процессы

Таблица 1 – Планирование и интегральные процессы

Процесс и роль	**Вход и начало**	**Результат и окончание**
Планирование, руководитель	Получены системные требования и сведения об уровне ПО. Для учебного проекта уровень фиксируют при согласовании области применения.	Планы и стандарты переданы на проверку. Переход разрешён после утверждения применимых редакций.
Верификация, независимый проверяющий	Известны проверяемая версия, основания, критерии и способ проверки.	Запись проверки и СП по отклонениям. Результат принят, когда критерии выполнены и открытых несоответствий по нему нет.
УК ПО, ответственный за УК	ЕК поступила на регистрацию либо получено СП или ЗИ.	Утверждённая версия и запись УК. Процесс для данной поставки завершён, когда по записи можно восстановить её состав.
Гарантия качества, проверяющий соблюдение плана	Есть применимый план, результаты этапа и записи о проверке.	Заключение о соблюдении плана и СП при отклонении. Замечания, влияющие на переход, закрыты.

Процесс и роль	Вход и начало	Результат и окончание
Взаимодействие с сертифицирующим органом, руководитель	В модели курса запрос и согласованный состав свидетельств открывают рассмотрение.	Комплект свидетельств, решение и замечания. Рассмотрение заканчивается после регистрации решения. В учебном проекте команда показывает документы преподавателю.

2.1 Разработка требований

Цель и вход. Аналитик описывает наблюдаемое поведение ПО по системным требованиям, архитектуре системы и программно-аппаратным интерфейсам. Два последних входа находятся в разделе 3 системных требований. Работой управляют настоящий план и Стандарт на разработку требований к ПО. Для первого прохода нужны проверенные входы и назначение исполнителя. Для изменения базовой версии нужен также утверждённый ЗИ.

Работа. Автор выделяет входы и выходы ПО. Например, сведения о сборке поступают от внешней системы, а результат регистрации версии доступен в каталоге. Для каждого результата автор формулирует требование ВУ, связывает его с системным требованием и задаёт способ проверки. Производное требование сопровождает обоснованием.

Выход и переход. Выходы — спецификация требований к ПО и сообщения о проблемах. Независимый проверяющий сверяет покрытие, содержание и связи по стандарту. После устранения замечаний утверждённую СПТ передают в УК и проектировщику. Место исходника по реестру документов — `docs/05_software_requirements.tex`, копии для чтения — каталог `pdf/`.

2.2 Проектирование

Цель и вход. Проектировщик получает утверждённую СПТ, План разработки и Стандарт на проектирование. При повторном входе получает ЗИ. Он начинает работу, когда известны базовая версия СПТ и область изменения.

Работа. Проектировщик распределяет требования между частями ПО, описывает их интерфейсы, состояния и требования НУ. Например, в проекте указывает, как сопоставить ответ о завершении установки с ранее выданным заданием. Для каждого требования НУ указывает родителя ВУ либо обоснование производного требования.

Выход и переход. Описание проекта включает архитектуру ПО и требования НУ. При несогласованности входов оформляют СП. Проверяющий сверяет интерфейсы обеих сторон обмена и покрытие требований ВУ. После утверждения УК регистрирует описание проекта в репозитории продукта и передаёт его версию разработчику.

2.3 Кодирование

Цель и вход. Разработчик получает описание проекта, План разработки, Стандарт на кодирование и при изменении — ЗИ. Работу начинает по утверждённой версии описания проекта с назначенным объёмом задания.

Работа. Разработчик реализует требования НУ, отдельно фиксирует параметры сборки и зависимости. Проверяющий сопоставляет изменения с описанием проекта и повторяет про-

верки выбранной версии. При расхождении автор регистрирует СП с указанием входной версии и наблюдаемого результата.

Выход и переход. Исходный код, параметры сборки и СП. После одобрения изменений в УК сохраняют идентификатор версии файлов и результаты проверки. Исполнитель интеграции получает именно эту версию.

2.4 Интеграция

Цель и вход. Исполнитель интеграции получает исходный код, параметры сборки, описание проекта, План разработки и ЗИ при изменении. Если конфигурация поставляется отдельно, в состав входов входит компонент параметрических данных. До начала он сверяет версии входов и готовность целевой среды.

Работа. Исполнитель собирает пакет из этих входов, загружает его на стенд и проверяет запуск. На имитаторе системы сборки передаёт известные сведения о версии компонента и сверяет результат регистрации с ожидаемым. Порядок действий дан в разделе 4.4.

Выход и переход. Исполняемый объектный код (в данном проекте — готовый к запуску пакет), протокол сборки, командные файлы сборки, протокол загрузки и СП. Ответственный за выпуск сверяет пакет и протоколы с манифестом по Плану УК. Только после совпадения контрольных сумм и успешной проверки запуска регистрирует готовность к выпуску. Открытое несоответствие блокирует передачу.

3 Организация и роли

Капитан команды — Заранкович Дмитрий Николаевич. В реестре учебных документов исполнители указаны отдельно для каждого файла. Фамилия автора документа сама по себе не определяет, кто будет разрабатывать, проверять или выпускать продукт.

Таблица 2 – Распределение работы

Работа	**Исполнитель**	**Ответственность**
Учебные документы	Брылкина К. К. — системные требования, Лазарев Я. О. — План УК, Пономаренко А. А. — План разработки и Стандарт требований.	Готовят редакцию своего документа к независимой вычитке.
Требования и проектирование	Аналитик и проектировщик, назначения по заданию.	Передают СПТ и описание проекта с исходными версиями.
Кодирование и интеграция	Разработчик и исполнитель интеграции, назначения по заданию.	Код и параметры передают на сборку. Пакет и протоколы передают на проверку.
Проверка и выпуск	Независимый проверяющий и ответственный за выпуск по Плану УК.	Проверяют результат и сверяют состав поставки.

[уточнить: Кого команда назначает на роли разработки, независимой проверки, УК, гарантии качества и выпуска продукта?]

4 Среда разработки

4.1 Документация требований

Автор ведёт исходник в LaTeX по шаблону проекта. Для передачи версии использует Git, для подготовки PDF — `build.sh`. Проверяющий сверяет текст и ссылки с исходными данными в той же зафиксированной редакции. Версия PDF указана в шапке.

4.2 Проектирование

Проектировщик ведёт описание архитектуры и требования НУ в версионируемых файлах. Для диаграммы сохраняет редактируемый исходник и изображение, для таблицы интерфейсов — отправителя, получателя, событие, данные и возможную ошибку. УК фиксирует комплект одним согласованным составом.

4.3 Разработка

Рабочий вариант для обсуждения — Python для серверной части: на нём можно описать переходы состояний и воспроизвести ответы внешних систем в испытаниях. Имитаторы должны выдавать также задержанные и ошибочные ответы. Выбор языка, версии среды и зависимостей фиксируют до первой базовой версии кода. [уточнить: Утверждает ли команда Python и поставку приложения с интерпретатором в составе образа?]

4.4 Интеграция, сборка и загрузка

Для рабочего варианта исполнителем команд служит задача непрерывной интеграции (CI), целевой средой — испытательный стенд. Пакет содержит программу и среду исполнения. Сборка самого продукта и внешняя система, передающая продукту сведения о сборках релизов, выполняют разные задачи.

1. Получить из УК точную версию кода, команды сборки, версии зависимостей и параметры. В протокол сборки записать значения входов.
2. Выполнить команды в чистой среде. Зафиксировать результат, идентификатор пакета и контрольную сумму. При ошибке приложить протокол к СП и остановить загрузку.
3. Взять пакет из зарегистрированной поставки. Сверить сумму с манифестом по Плану УК. Записать исходное состояние стенда, развернуть пакет и прочитать фактическую версию и сумму на стенде.
4. Сопоставить значения с манифестом, проверить запуск и регистрацию версии на имитаторе. В протокол загрузки внести стенд, время, версию, исполнителя и результат. При расхождении остановить передачу, зарегистрировать СП и вернуть стенд к последней проверенной поставке.

Для повторной сборки другой исполнитель использует командные файлы, версии входов и параметры из протокола. Место публикации пакета и протоколов фиксируют по Плану УК.

5 Стандарты

Стандарт на разработку требований к ПО определяет карточку, трассировку и проверку СПТ. Стандарт на проектирование задаёт правила описания частей ПО, интерфейсов и требований НУ. Стандарт на кодирование задаёт правила исходного текста, обработки ошибок и

оформления параметров сборки. Два последних документа предстоит подготовить к соответствующим процессам. Вход в процесс разрешают после утверждения применимого стандарта.

Лист регистрации изменений

Версия	Дата	Что изменено	Кто	Основание
1	13.11.2026	Первая версия документа	Пономаренко А. А.	Задание к паре 31.10.2026